Sentinel: Hardware Roadmap

OEM Ring Procurement + Integration Plan (M1-M3)

For Samsung Solve for Tomorrow IRIS ISEF

Sentinel ? Hardware Integration Roadmap

Funding: Emergent Ventures grant. Hardware (wearable ring with vendor SDK/API access) arriving soon.

Goal

Get real sensor data flowing from a physical ring into Sentinel's clinical and crisis pipelines, replacing simulated data with a real, secure ingestion path ? without re-architecting anything downstream.

Architecture decision

Hardware: OEM ring from Jport (China) ? not a consumer ring with a public SDK.
OEM rings expose one of three interfaces; the adapter depends on which one:

1. BLE GATT (most common for OEM rings) ? the ring advertises a custom BLE service; the vendor doc lists service UUID + characteristic UUIDs (HR, stress, battery, ...) and the byte format for each. Bridge = BLEGATTRingSource (bleak) on a PC/phone gateway.
2. Vendor app / SDK (Android/iOS companion) ? data is read through the OEM's app; bridge = vendor SDK adapter (VendorAPIRingSource) or app data export.
3. Cloud API ? the ring syncs to the vendor cloud; bridge = VendorAPIRingSource polling the vendor REST API.

All three converge on the same downstream path:

`
ring --(vendor interface)--> RingSource adapter --> POST /ring/data --> DB + pipelines (in app/services/ring/)
`

Everything downstream (dashboards, discrepancy engine, crisis, risk) already consumes RingSensorLog / SensorReading, so the only new surface is *ingestion*.

Current state (M0 done)

- backend/app/services/ring/ ? pluggable SDK layer:
- base.py ? RingSource contract + SensorData
- simulated.py ? deterministic per-user/per-hour SimulatedRing (calm/balanced/stressed)
- vendor_api.py ? VendorAPIRingSource adapter base for vendor SDK/API
- ring_devices table + device binding:
- POST /ring/pair ? bind serial ? patient, issue one-time device token (SHA-256 hashed at rest)
- POST /ring/unpair ? revoke device (re-pairing re-issues a token)
- GET /ring/devices ? list devices (patient + psych)
- POST /ring/data ? accepts device-token auth (X-Device-Serial + X-Device-Token) or patient JWT fallback
- 401 on wrong token / unknown serial / revoked device; last_seen_at tracked

- scripts/sim_ring.py ? streams simulated readings via the device-token path
- scripts/ring_bridge.py ? generic bridge: any RingSource ? POST /ring/data
- backend/app/services/ring/ble_gatt.py ? BLEGATTRingSource (bleak, optional dep):
standard HRM parser (0x2A37), configurable char map + parser hooks, battery read,
override point for OEM proprietary blobs
- backend/requirements-bridge.txt ? bleak (bridge-only dep, not the API)
- scripts/test_ring_api.py ? SDK unit tests, incl. BLE parsers (all passing)

Verified E2E: pair ? device-token push ? stored log ? last_seen_at updated ? 401s for bad auth.

Milestones

M1 ? First live device (when the Jport ring + spec arrives)

- Get the Jport doc (spec request sheet below); confirm interface (BLE GATT / app SDK / cloud API).
- Fill in the adapter (skeleton already built):
- BLE: configure BLEGATTRingSource UUIDs (--hrm-uuid, --hrv-uuid, --stress-uuid) + byte parsers for proprietary characteristics.
- App/Cloud: subclass VendorAPIRingSource, implement _fetch().
- Run scripts/ring_bridge.py with the real source; verify readings land in /ring/data.
- Offline buffering: intermittent vendor data ? queue readings locally, flush in order on reconnect.
- Device health surfaced: battery, signal, last_seen_at in GET /ring/devices; "ring status" chip in UI.

Spec request sheet (send to Jport)

Before M1 can be finished, ask the vendor for:
1. BLE service UUID(s) and, for each metric (HR, HRV, stress, sleep, SpO2, battery), the characteristic UUID + whether it's notify or read.
2. Byte-level data format per characteristic (little-endian? units? e.g. HR = 1 byte in bpm).
3. Sampling/streaming rate ? does it push continuously (notify) or only on request?
4. Sleep data: does the ring expose raw sleep periods or only aggregate hours?
5. Battery: is there a battery service (standard 0x180F) or custom?
6. Pairing: any PIN/bonding requirements; does it pair with any BLE host or only the vendor app?
7. If a companion app exists: does it expose data locally (BLE from the phone) or only via vendor cloud?
8. Cloud API (if any): REST endpoints, auth model, rate limits, raw HR/HRV access (many OEM clouds only expose aggregated wellness scores ? confirm raw signals are accessible).

M2 ? Live vitals + automatic safety pipelines

- WebSocket stream of live vitals to the psych dashboard (extend existing websocket_manager).
- Auto-discrepancy: on journal save, auto-pull the patient's latest ring readings and run /discrepancy/check (remove manual trigger).
- Biometric crisis triggers: sustained stress > 80, HR > 130, HRV < 20 ? feed the crisis/escalation engine with a cooldown to avoid alert spam.

M3 ? Pilot readiness (target: Sept 2026, 30 rings)

- Pair-onboarding UX for patients (bind ring via code/QR during onboarding).
- Device registry + per-device uptime/battery monitoring for psychs.
- Dedup, gap/backfill handling for intermittent uploads.
- Docker Compose ring-bridge service; device-token rotation/revocation tooling.
- E2E test: simulated stream ? stored ? discrepancy alert ? crisis flow.

Security notes

- Device tokens are SHA-256 hashed at rest; validated with constant-time compare.
- A ring never holds a patient password ? it authenticates with its own token.
- Revoking a device kills its token immediately (status check on every push).
- Vendor cloud API credentials (Oura/Ultrahuman OAuth) live in the bridge config, never in the DB.

Open questions before M1

1. Which interface does Jport's ring expose ? BLE GATT, companion app SDK, or cloud API? (determines the adapter)
2. If BLE: do we get the full service/characteristic UUID map + byte format, or only the vendor app?
3. If cloud API: do they expose raw HR/HRV/SpO2 signals, or only aggregated wellness scores?
4. Where does the bridge run ? clinic LAN gateway (Raspberry Pi / PC) or the user's phone?